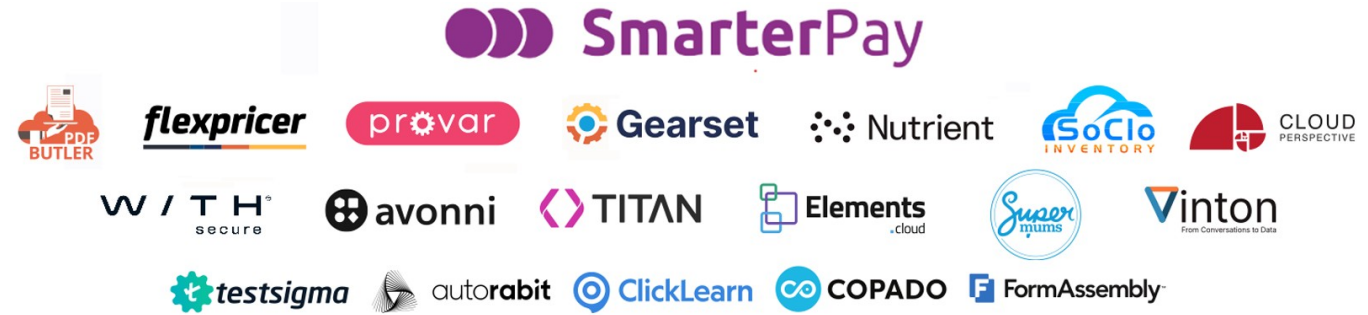




Scaling Apex and LWC with Apex Cursors

Andrew Fawcett, AndyInTheCloud Consulting Ltd

andy@andyinthecloud.com

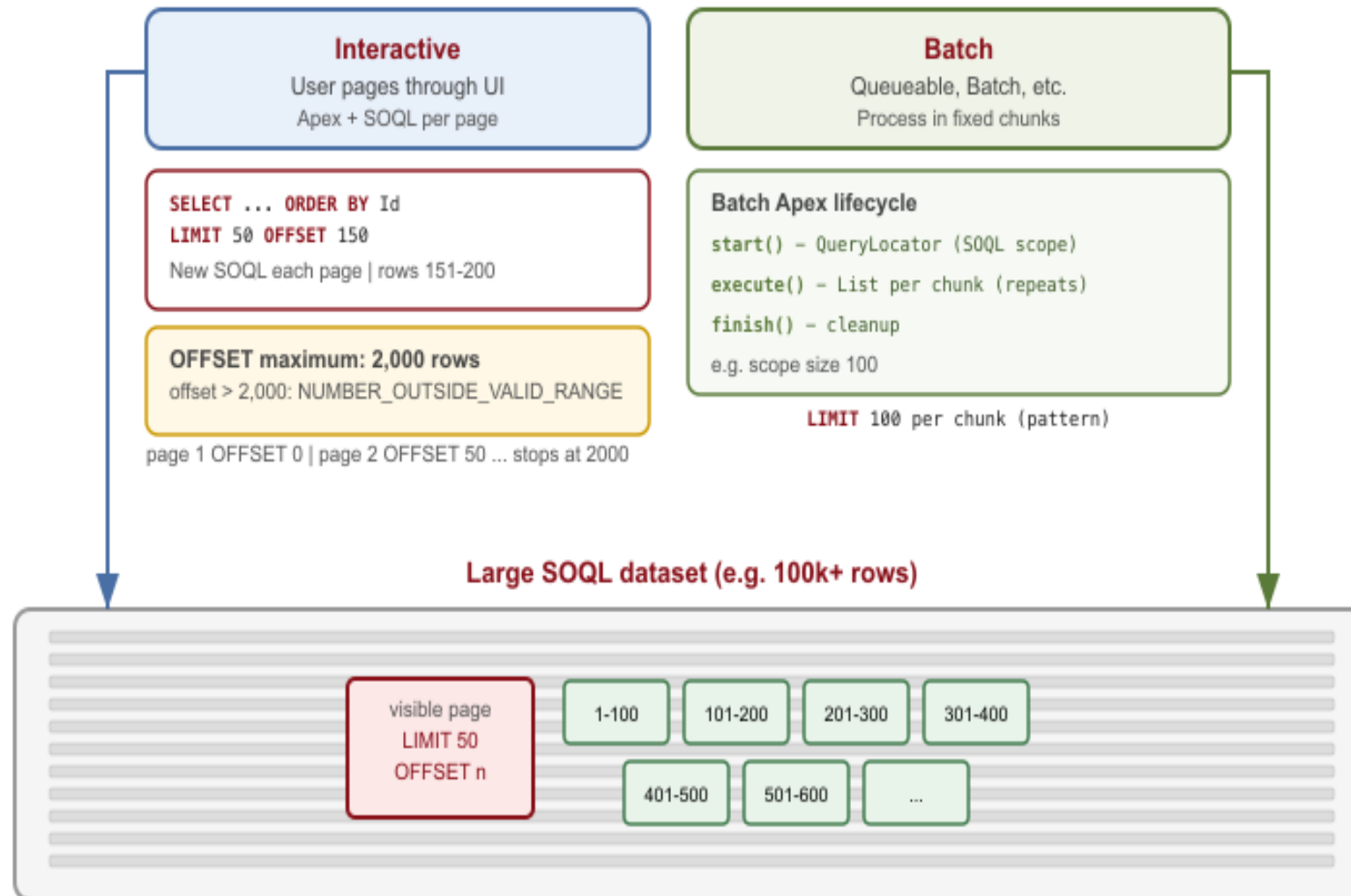
<https://www.linkedin.com/in/andyfawcett/>

#LDNsCall #LC26



Accessing and processing datasets today in SOQL

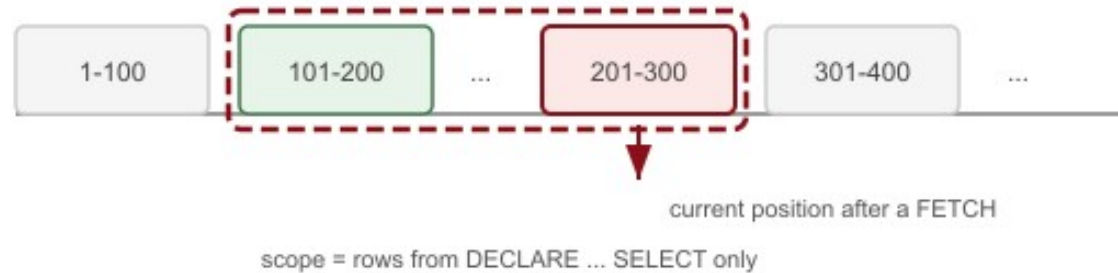
UI paging with OFFSET/LIMIT vs async processing in fixed-size chunks.



What is a SQL Cursor?

Traverse a query result incrementally instead of returning the full set at once.

1. One declared result set - position moves along rows



Sections 2 and 3 = two views of the same idea

2 = lifecycle statements (DECLARE ... CLOSE)

3 = access patterns (how FETCH moves in that scope)

Different from OFFSET paging (new query per page)

2. Lifecycle pattern (SQL statements)

Ordered steps the application executes

- 1 **DECLARE** c **CURSOR FOR** <select>;
Defines query scope
- 2 **OPEN** c;
Before first row
- 3 **FETCH** ... **FROM** c **INTO** ...; (repeat)
NEXT, PRIOR, ABSOLUTE...
- 4 **CLOSE** c;
Release resources
- 5 **DEALLOCATE** c; (some products)
Drop definition

Example

```
DECLARE account_cur CURSOR FOR
SELECT id, name FROM accounts
WHERE active ORDER BY id;

OPEN account_cur;
FETCH NEXT FROM account_cur ...
CLOSE account_cur;
```

3. Access patterns (how you read rows)

FETCH orientations - same open cursor, move position in scope

Forward-only cursor (typical default)

FETCH NEXT - one row forward
Most common in ETL / streaming loops

Scrollable cursor (when supported)

FETCH NEXT | **PRIOR** | **FIRST** | **LAST**

FETCH ABSOLUTE n | **FETCH RELATIVE** n

Jump relative to start or current position (product-dependent)

Not a cursor access pattern:

SELECT ... OFFSET 20 FETCH 10

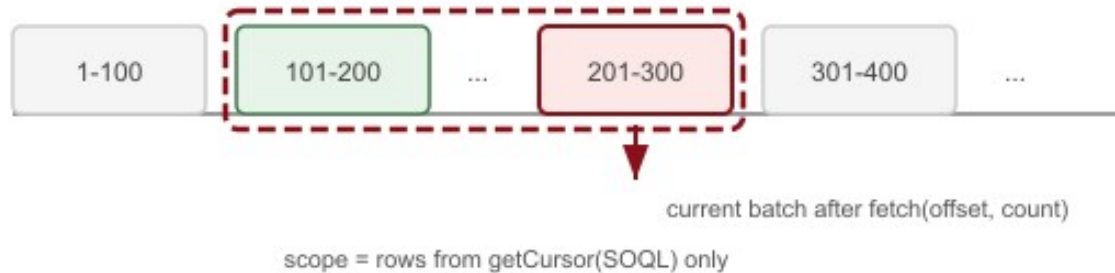
new query each page - no held position
cursor: one OPEN, many FETCH in same scope



What is a SOQL Cursor?

SQL cursor concepts on Salesforce: Database.getCursor(SOQL) and incremental fetch in Apex.

1. One SOQL result set - position moves along rows



Apex maps SQL cursor ideas to API calls

2 = lifecycle (getCursor, fetch, hold state)

3 = access patterns (offset batch vs fetchPage)

Not [SELECT ...] loading all rows at once

2. Lifecycle pattern (Apex API)

No DECLARE/OPEN/CLOSE - platform cursor over SOQL

- 1 Database.getCursor(soql);
Or PaginationCursor
- 2 cursor.getNumRecords();
Optional row count
- 3 cursor.fetch(offset, count);
Repeat until done
- 4 Hold cursor: LWC or Cache
Available across requests
- 5 Governed by Limits.*ApexCursor*
Apex cursor limits

Example (this repo)

```
Database.Cursor c =
Database.getCursor(
    'SELECT Id, Name FROM Account
    WHERE Name LIKE 'TEST%'
    ORDER BY Name',
    AccessLevel.USER_MODE);
List<Account> batch =
    c.fetch(offset, pageSize);
```

3. Access patterns (how you read rows)

Incremental reads inside one open SOQL scope

Database.Cursor (standard)

fetch(offset, count)
Forward by offset into scope (infinite scroll)

Database.PaginationCursor (GA)

fetchPage(start, pageSize)
Page-oriented access; CursorFetchResult

Not a SOQL cursor pattern:

List<SObject> = [SELECT ...]

position in scope

0-99 100-199 ... 200-299

fetch(200, 100) = rows 200-299

all rows in heap at once
cursor: one SOQL scope, many fetches

Usecases for Apex Cursors

Interactive

- Analyst large data view
- Reporting and drilldown
- Preview job records to process

```
apex>  
Database.PaginationCursor c =  
    Database.getPaginationCursor(  
        soql, AccessLevel.USER_MODE);  
Database.CursorFetchResult r =  
    c.fetchPage(start, pageSize);
```

Batch

- Orchestrate process execution
- Look ahead to optimize resource usage
- Safer immutable record set

```
apex>  
Database.Cursor c =  
    Database.getCursor(  
        soql, AccessLevel.USER_MODE);  
List<Account> batch =  
    c.fetch(offset, pageSize);
```

Architecture

24hr cached large record sets | Scalable replacement to SOQL OFFSET | Bidirectional navigation

Demo 1: Virtual Data Table



Demo 1: Highlights



Demo 2: Interactive Reporting and Drilldown



Demo 2: Highlights



Demo 3: Adaptive Async



Demo 3: Highlights

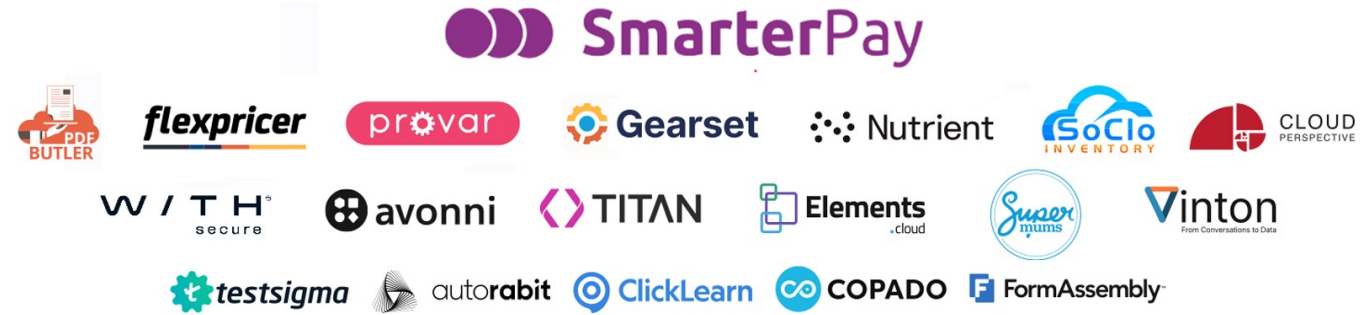


Limits



Tips and Tricks





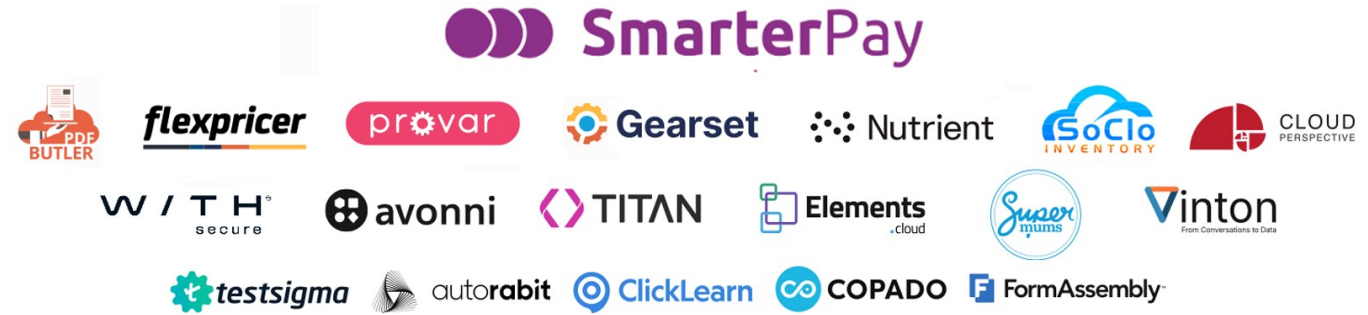
Q&A

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn

#LDNsCall #LC26





Thank You

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn

#LDNsCall #LC26

